The code below trains a three-layer feedforward neural network model using the "Human Activity Recognition Using Smartphones" dataset.

Question 1)

-CODE Explanation

```
clear
clc

% Load training and test data from the respective files to MATLAB
X_train = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\
UCI HAR Dataset\train\X_train.txt");
Y_train = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\
UCI HAR Dataset\train\y_train.txt");
X_test = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\UCI
HAR Dataset\test\x_test.txt");
Y_test = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\UCI
HAR Dataset\test\y_test.txt");

% Define parameters for the artificial neural network
h1_size = 200; % Number of neurons in the first hidden layer
h2_size = 100; % Number of neurons in the second hidden layer
epochLimit = 80; % Epoch limit
input_size = size(X_train, 2); % Number of inputs in the X_train data set (561
inputs)
Wh1e = (rand(h1_size, input_size + 1) - 0.5) / 100; % Initial weights for the
first hidden layer
Wh2e = (rand(h2_size, h1_size + 1) - 0.5) / 100; % Initial weights for the
second hidden layer
Woe = (rand(length(unique(Y_test)), h2_size + 1) - 0.5) / 100; % Initial
weights for the output layer

wo = Woe(:, :); % All weights for the output layer are assigned to w0
w1 = Wh1e(:, :); % All weights for the first hidden layer are assigned to w0
w2 = Wh2e(:, :); % All weights for the second hidden layer are assigned to w0

% Add a bias term to the input data
X_train = [X_train() ones(7352, 1)]; % Bias value of 1 is added to the X_train
input data (7352 times)
X_test = [X_test() ones(2947, 1)]; % Bias value of 1 is added to the X_test
input data (2947 times)

% Variables for activity numbers are defined. These variables represent the
output classes and will be used in the confusion matrix.
yurume=0; yukari_yurume=0; asagi_yurume=0; oturma=0; ayakta=0; yatma=0;
yurume_tr=0; yukari_yurume_tr=0; asagi_yurume_tr=0; oturma_tr=0; ayakta_tr=0;
yatma_tr=0;

% Variables for unsuccessful cases in the confusion matrix for training and
test sets are defined.
k1 = 0; k2 = 0; k3 = 0; k4 = 0; k5 = 0; k6 = 0; k7 = 0; k8 = 0; k9 = 0; k10 =
0; k11 = 0; k12 = 0; k13 = 0; k14 = 0; k15 = 0;
k16 = 0; k17 = 0; k18 = 0; k19 = 0; k20 = 0; k21 = 0; k22 = 0; k23 = 0; k24 =
0; k25 = 0; k26 = 0; k27 = 0; k28 = 0; k29 = 0; k30 = 0;
p1 = 0; p2 = 0; p3 = 0; p4 = 0; p5 = 0; p6 = 0; p7 = 0; p8 = 0; p9 = 0; p10 =
0; p11 = 0; p12 = 0; p13 = 0; p14 = 0; p15 = 0;
```

```
p16 = 0; p17 = 0; p18 = 0; p19 = 0; p20 = 0; p21 = 0; p22 = 0; p23 = 0; p24 =
0; p25 = 0; p26 = 0; p27 = 0; p28 = 0; p29 = 0; p30 = 0;

% Empty arrays are defined to save mean square error data.
mse_matrix_tr = [];
mse_matrix_test = [];
mse_matrix = [];
min_mse_matrix_tr = [];
min_mse_matrix_test = [];

% Training loop starts here using Stochastic Gradient Descent to update
parameters
for epoch = 1:80 % Training continues until the epoch limit is reached
    epoch % Display which epoch the training is at
    for i = 1:7352 % Training is performed for 7352 input patterns. A total of
588160 iterations are performed (80(epoch) * 7352(pattern))
        x1 = X_train(i, :); % All inputs in the X_train data set are taken
        z1 = x1 * transpose(w1); % All inputs are multiplied by the transpose
of weights for the first hidden layer to find the outputs of the first hidden
layer neurons.
        h1 = tanh(z1); % The obtained z1 output is passed through the tanh
activation function to determine the h1 parameter.
        h1 = [h1 1]; % An additional 1 input is added to the h1 parameter for
the bias value for the second hidden layer.
        y1 = h1 * transpose(w2); % The h1 input is multiplied by the transpose
of weights for the second hidden layer to find the outputs of the second hidden
layer neurons.
        r1 = ReLU(y1); % The obtained y1 values are applied with the ReLU
activation function. r1 parameter is determined.
        r1 = [r1 1]; % A bias input value is added to the r1 parameter for the
output layer.
        ow1 = r1 * transpose(wo); % The r1 input is multiplied by the transpose
of weights for the output layer to find the outputs of the output layer
neurons.
        o1 = sigmoid(ow1); % The obtained ow1 values are passed through the
sigmoid activation function. o1 parameter is determined. Thus, neural network
outputs are obtained.

        % Encode the real labels
        df = encoding(Y_train); % Y_train output is one-hot encoded.
        d = df(i, :); % All one-hot encoded outputs are assigned to the d
parameter.

        % Calculate error and learning rate
        e = d - o1; % The error between the desired value and the output of the
model is calculated.
        lr = 0.001; % Learning rate value is defined.

        % Backpropagation
        sigma_kl = diag(dersigmoid(ow1)) * transpose(e); % Weight change for
the output layer is calculated
        delta_wo = lr * (sigma_kl * r1);
        wo = wo + delta_wo; % Weight change is added to the weights from the
previous iteration to find new weights.

        sigma_kl2 = diag(derReLU(y1)) * (wo(:, 1:100))' * sigma_kl; % Weight
change for the second hidden layer is calculated
        delta_w2 = lr * sigma_kl2 * h1;
        w2 = w2 + delta_w2; % Weight change is added to the weights from the
previous iteration to find new weights.
```

```
        sigma_kl1 = diag(dertanh(z1)) * (w2(:, 1:200))' * sigma_kl2; % Weight
change for the first hidden layer is calculated
        delta_w1 = lr * sigma_kl1 * x1;
        w1 = w1 + delta_w1; % Weight change is added to the weights from the
previous iteration to find new weights.

        % Determine the index of the largest value in the results for training
data
        o_train_max(i) = max(o1, [], 2);
        o_train_index(i) = find(o1 == o_train_max(i));
    end

    % Calculate mean square error for training data
    mse_train = (1/(2 * 7352)) * (e).^2;
    mse_matrix_tr = [mse_matrix_tr, mse_train];
    min_mse_matrix_tr(epoch) = min(mse_matrix_tr);

    % Start the test loop. The same operations as in the training part are
performed. However, the weight update part is done here.
    for i = 1:2947
        x1 = X_test(i, :);
        z1 = x1 * transpose(w1);
        h1 = tanh(z1);
        h1 = [h1 1];
        y1 = h1 * transpose(w2);
        r1 = ReLU(y1);
        r1 = [r1 1];
        ow1 = r1 * transpose(wo);
        o1 = sigmoid(ow1);

        % Encode the real labels
        df = encoding(Y_test); % Y_test output is one-hot encoded.
        d = df(i, :); % All one-hot encoded outputs are assigned to the d
parameter.

        % Determine the index of the largest value in the results for test data
        et = d - o1;
        o_test_max(i) = max(o1, [], 2);
        o_test_index(i) = find(o1 == o_test_max(i));
    end

    % Calculate mean square error for test data
    mse_test = (1/(2 * 2947)) * (et).^2;
    mse_matrix_test = [mse_matrix_test, mse_test];
    min_mse_matrix_test(epoch) = min(mse_matrix_test);

end

% Plot the mean square error graph
figure;
plot(1:epoch, min_mse_matrix_tr, 1:epoch, min_mse_matrix_test);
grid;
xlabel('epoch');
ylabel('MSE');
title('Figure 1 - Neural Network');

% Determine confusion matrices for training and test data
```

```
%In this section, an attempt is made to obtain the confusion matrix for the
training dataset. The confusion matrix is a matrix used to evaluate the
performance of a classification model. This matrix illustrates the relationship
between the classification decisions made by the model and the actual class
labels. The confusion matrix is commonly used in classification problems, where
each row represents the actual classes and each column represents the predicted
classes. Each if block written in this section corresponds to 36 parameters
belonging to a 6x6 confusion matrix, considering that there are 6 classes. The
numbers on the diagonals of the confusion matrix indicate how many times the
model correctly predicted the outputs. The if blocks examine whether the
model's output matches the dataset's output. Then, by examining the output
value of the model and the output value of the dataset, the class to which it
belongs is determined. Taking these conditions into account, the number of
parameters in the appropriate position in the matrix is increased. Thus, the
confusion matrix is obtained.

for j=1:7352
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==1)
                    yurume_tr=yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==2)
                    yukari_yurume_tr=yukari_yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==3)
                    asagi_yurume_tr=asagi_yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==4)
                    oturma_tr=oturma_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==5)
                    ayakta_tr=ayakta_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==6)
                    yatma_tr=yatma_tr+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==1)
                    p1=p1+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==1)
                    p2=p2+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==1)
                    p3=p3+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==1)
                    p4=p4+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==1)
                    p5=p5+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==2)
                    p6=p6+1;
```

```
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==2)
                    p7=p7+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==2)
                    p8=p8+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==2)
                    p9=p9+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==2)
                    p10=p10+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==3)
                    p11=p11+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==3)
                    p12=p12+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==3)
                    p13=p13+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==3)
                    p14=p14+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==3)
                    p15=p15+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==4)
                    p16=p16+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==4)
                    p17=p17+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==4)
                    p18=p18+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==4)
                    p19=p19+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==4)
                    p20=p20+1;
                end
```

```
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==5)
                    p21=p21+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==5)
                    p22=p22+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==5)
                    p23=p23+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==5)
                    p24=p24+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==5)
                    p25=p25+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==6)
                    p26=p26+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==6)
                    p27=p27+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==6)
                    p28=p28+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==6)
                    p29=p29+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==6)
                    p30=p30+1;
                end
            end
% Confusion matrix for training
confusionMatrixTraining=[yurume_tr p6 p11 p16 p21 p26;
        p1 yukari_yurume_tr p12 p17 p22 p27;
        p2 p7 asagi_yurume_tr p18 p23 p28;
        p3 p8 p13 oturma_tr p24 p29;
        p4 p9 p14 p19 ayakta_tr p30;
        p5 p10 p15 p20 p25 yatma_tr];


%In this section, an attempt is made to obtain the confusion matrix for the
test dataset. The confusion matrix is a matrix used to evaluate the performance
of a classification model. This matrix illustrates the relationship between the
classification decisions made by the model and the actual class labels. The
confusion matrix is commonly used in classification problems, where each row
represents the actual classes and each column represents the predicted classes.
Each if block written in this section corresponds to 36 parameters belonging to
a 6x6 confusion matrix, considering that there are 6 classes. The numbers on
the diagonals of the confusion matrix indicate how many times the model
```

correctly predicted the outputs. The if blocks examine whether the model's output matches the dataset's output. Then, by examining the output value of the model and the output value of the dataset, the class to which it belongs is determined. Taking these conditions into account, the number of parameters in the appropriate position in the matrix is increased. Thus, the confusion matrix is obtained.

```
        for j=1:2947
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==1)
                    yurume=yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==2)
                    yukari_yurume=yukari_yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==3)
                    asagi_yurume=asagi_yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==4)
                    oturma=oturma+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==5)
                    ayakta=ayakta+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==6)
                    yatma=yatma+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==1)
                    k1=k1+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==1)
                    k2=k2+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==1)
                    k3=k3+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==1)
                    k4=k4+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==1)
                    k5=k5+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==2)
                    k6=k6+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==2)
                    k7=k7+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==2)
```

```
                    k8=k8+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==2)
                    k9=k9+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==2)
                    k10=k10+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==3)
                    k11=k11+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==3)
                    k12=k12+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==3)
                    k13=k13+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==3)
                    k14=k14+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==3)
                    k15=k15+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==4)
                    k16=k16+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==4)
                    k17=k17+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==4)
                    k18=k18+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==4)
                    k19=k19+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==4)
                    k20=k20+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==5)
                    k21=k21+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==5)
                    k22=k22+1;
```

```
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==5)
                    k23=k23+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==5)
                    k24=k24+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==5)
                    k25=k25+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==6)
                    k26=k26+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==6)
                    k27=k27+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==6)
                    k28=k28+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==6)
                    k29=k29+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==6)
                    k30=k30+1;
                end
            end
% Confusion matrix for test
confusionMatrixTest=[yurume k6 k11 k16 k21 k26;
        k1 yukari_yurume k12 k17 k22 k27;
        k2 k7 asagi_yurume k18 k23 k28;
        k3 k8 k13 oturma k24 k29;
        k4 k9 k14 k19 ayakta k30;
        k5 k10 k15 k20 k25 yatma];
% Calculate success rates
trainingSuccessRate =
sum(diag(confusionMatrixTraining))/sum(sum(confusionMatrixTraining));
testSuccessRate = sum(diag(confusionMatrixTest))/sum(sum(confusionMatrixTest));

% Function for one-hot encoding
function o = encoding(v)
    % Giriş vektörünün uzunluğunu al
    n = length(v);

    % Çıkış matrisini başlangıçta sıfırlarla doldur
    o = zeros(n, 6);

    % Her eleman için encoding işlemi yap
    for i = 1:n
        switch v(i)
            case 1
                o(i, :) = [1 0 0 0 0 0];
```

```
            case 2
                o(i, :) = [0 1 0 0 0 0];
            case 3
                o(i, :) = [0 0 1 0 0 0];
            case 4
                o(i, :) = [0 0 0 1 0 0];
            case 5
                o(i, :) = [0 0 0 0 1 0];
            case 6
                o(i, :) = [0 0 0 0 0 1];
            otherwise
                error('Geçersiz giriş değeri');
        end
    end
end

% Sigmoid activation function
function[o] = sigmoid(v)
    o=1./((exp(-v))+1);
end
% Tanh activation function
function[o] = tanh(v)
    o=(exp(2*v)-1)./(exp(2*v)+1);
end
% RELU activation function
function[a] = ReLU(b)
    for i=1:100
        if(b(i)<0)
            a(i)=0;
        elseif(b(i)>=0)
            a(i)=b(i);
        end
    end
end

% Derivative of the sigmoid activation function
function[o] = dersigmoid(v)
    o=(1./((exp(-v))+1)).*(1-(1./((exp(-v))+1)));
end

% Derivative of the ReLU activation function
function[a] = derReLU(b)
    for i=1:100
        if(b(i)<0)
            a(i)=0;
        elseif(b(i)>=0)
            a(i)=1;
        end
    end
end

% Derivative of the tanh activation function
function[o] = dertanh(v)                        % Activation Function
    o=1-(((exp(2*v)-1)./(exp(2*v)+1)).^2);
end
```

This code trains a three-layer feedforward neural network model using the "Human Activity Recognition Using Smartphones" dataset. The dataset consists of a total of 10,299 movement

patterns with 561 features and 6 classes. It is divided into separate subsets for training and testing. The neural network comprises 200 and 100 neurons in the first and second hidden layers, respectively. Training is performed using stochastic gradient descent (SGD) with a learning rate set to 0.001. Predictions are made during training using the updated weights through the backpropagation method, and the mean squared error is calculated. The training process continues for 80 epochs. Additionally, training and test results are evaluated using success rates and confusion matrices. Important parameters such as training, learning rate, and model performance are meticulously monitored through loops and matrices. The performance of the trained model is visualized through the mean squared error graph and success rates obtained after training.

-Graph of the variation of Mean Squared Error (MSE) loss with respect to the number of epochs on training and test sets

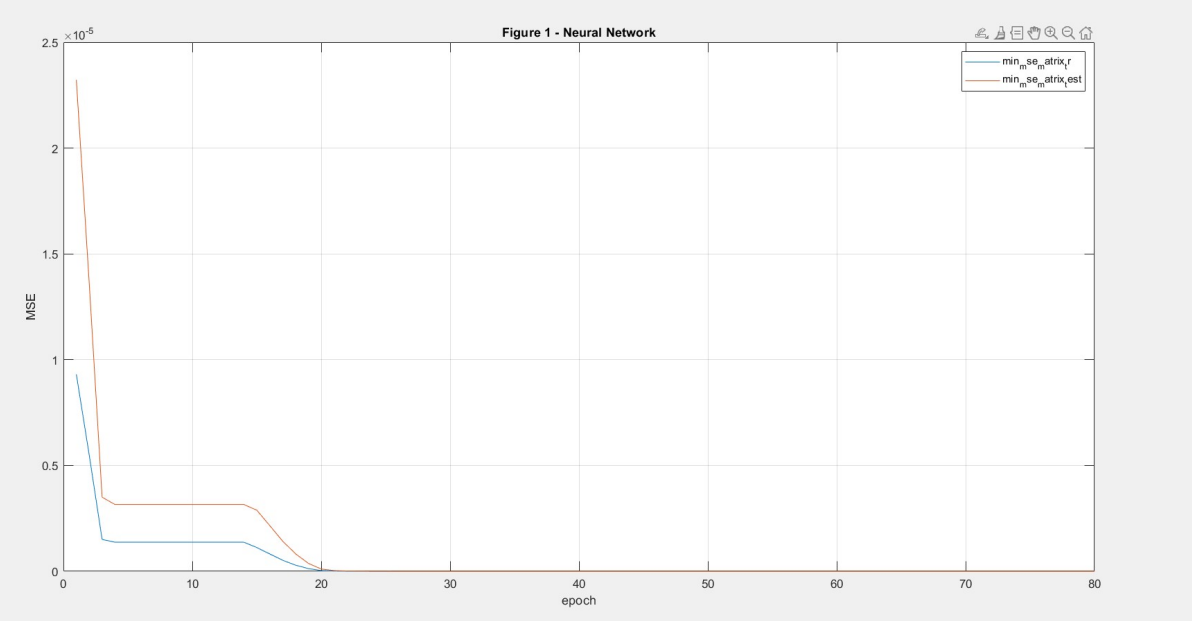


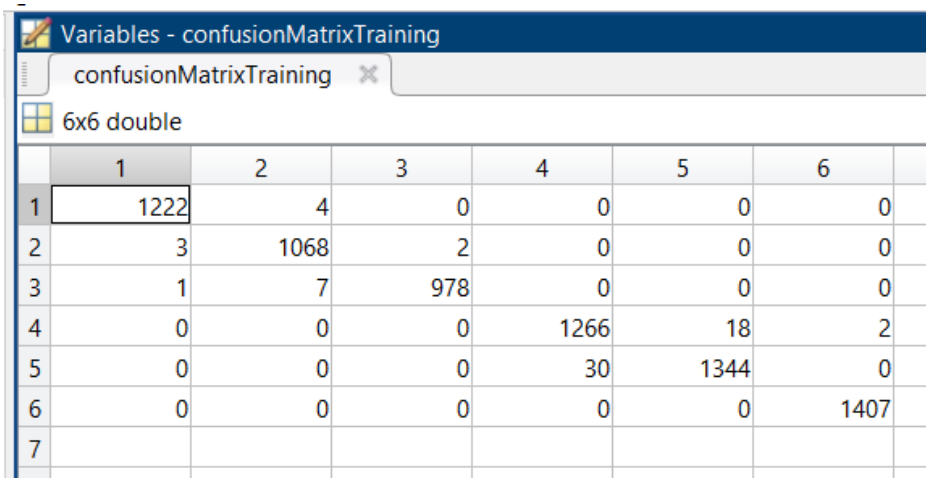
Figure 1:MSE Graph

The minimum loss value obtained and the epoch number at which this value occurred for each set. Starting from the top row and going down, the minimum loss values for all epochs from 1 to 80 are provided below

2.32261418724402e-05	1.36235205340678e-05
3.50309958914228e-06	3.15636641283276e-06
3.15636641283276e-06	3.15636641283276e-06
3.15636641283276e-06	3.15636641283276e-06
3.15636641283276e-06	3.15636641283276e-06
3.15636641283276e-06	3.15636641283276e-06
3.15636641283276e-06	3.15636641283276e-06
2.88835925972517e-06	2.16600240405226e-06
1.42948083686857e-06	8.30407063535570e-07
3.77525649462534e-07	1.07315438232245e-07
3.39306867483772e-08	1.49018810313015e-08
7.58505540685969e-09	4.13853758105429e-09
2.38859969071741e-09	1.48378134182052e-09
9.95912489862396e-10	7.02260818291936e-10
5.11312169853847e-10	3.70458397433389e-10
2.55930338954162e-10	1.84700782770693e-10

1.39636049420980e-10	1.07753613468672e-10
8.63007601796001e-11	3.90564275683356e-11
1.81464758669329e-11	8.97223145525379e-12
4.68461774614558e-12	2.62057468187867e-12
1.57291010008615e-12	9.99700464238960e-13
6.80956225898404e-13	4.89608982109640e-13
3.70679965377657e-13	2.80354498714374e-13
2.05573784338569e-13	1.48504650697601e-13
9.79209415411126e-14	5.57867582446153e-14
3.13639989848115e-14	1.98977593561532e-14
1.42509090295668e-14	1.11910747809443e-14
9.10188257920040e-15	7.67516047370098e-15
6.43591208813319e-15	5.48968835640511e-15
4.69901924972680e-15	3.97825387471563e-15
3.35346248564554e-15	2.83843832462903e-15
2.39688121560798e-15	2.04747399329187e-15
1.76579402956484e-15	1.50998929720222e-15
1.29154668102327e-15	1.10983138721724e-15
9.51464795638020e-16	8.21690239646380e-16
7.09020936621967e-16	6.14620705652253e-16
5.32732700181757e-16	4.61397808739030e-16
3.96167084061300e-16	3.40606482177248e-16
2.92008120280114e-16	2.49911971224345e-16
2.11816399477312e-16	1.80937809081075e-16

-Using the weights obtained at the end of training, the confusion matrix for the training set is as follows:



	1	2	3	4	5	6
1	1222	4	0	0	0	0
2	3	1068	2	0	0	0
3	1	7	978	0	0	0
4	0	0	0	1266	18	2
5	0	0	0	30	1344	0
6	0	0	0	0	0	1407
7						

Figure 2:Confusion Matrix for Training

-Using the weights obtained at the end of training, the confusion matrix for the test set is as follows:

Variables - confusionMatrixTest

confusionMatrixTest

6x6 double

	1	2	3	4	5	6	7
1	484	0	12	0	0	0	
2	17	451	3	0	0	0	
3	22	21	377	0	0	0	
4	0	0	0	447	44	0	
5	0	0	0	31	501	0	
6	0	0	0	25	0	512	
7							

Figure 3:Confusion Matrix for Test

-The calculated success rate for the training set is as follows:

Variables - trainingSuccessRate

trainingSuccessRate

1x1 double

	1	2	3
1	0.9902		
2			
3			
4			

Figure 4:Success Rate of Training

-The calculated success rate for the test set is as follows:

Variables - testSuccessRate

testSuccessRate

1x1 double

	1	2	3
1	0.9413		
2			
3			

Figure 5:Success Rate of Test

Question 2)

In this question, the softmax function is utilized instead of the sigmoid function, and the cross-entropy cost function is used in place of the mean squared error. Furthermore, the values requested in the first question are recalculated for the second question. The updated code for the second question is provided below. As the code was explained in detail in the first question, it is not further elaborated in this section.

-CODE

```
% CE tamam softmax hazır toolbox ama softmax türevi sıkıntı

clear
clc

X_train = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\
UCI HAR Dataset\train\X_train.txt");
Y_train = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\
UCI HAR Dataset\train\y_train.txt");
X_test = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\UCI
HAR Dataset\test\x_test.txt");
Y_test = load("human+activity+recognition+using+smartphones\UCI HAR Dataset\UCI
HAR Dataset\test\y_test.txt");

h1_size = 200;
h2_size = 100;
epochLimit = 80;
input_size = size(X_train,2);
Wh1e = (rand(h1_size, input_size + 1)-0.5)/100;
Wh2e = (rand(h2_size, h1_size + 1)-0.5)/100;
Woe = (rand(length(unique(Y_test)), h2_size + 1)-0.5)/100;


wo=Woe(:,:);
w1=Wh1e(:,:);
w2=Wh2e(:,:);

X_train=[X_train() ones(7352,1)];
X_test=[X_test() ones(2947,1)];

yurume=0;
yukari_yurume=0;
asagi_yurume=0;
oturma=0;
ayakta=0;
yatma=0;

yurume_tr=0;
yukari_yurume_tr=0;
asagi_yurume_tr=0;
oturma_tr=0;
ayakta_tr=0;
yatma_tr=0;

k1=0;k2=0;k3=0;k4=0;k5=0;k6=0;k7=0;k8=0;k9=0;k10=0;k11=0;k12=0;k13=0;k14=0;k15=
0;
k16=0;k17=0;k18=0;k19=0;k20=0;k21=0;k22=0;k23=0;k24=0;k25=0;k26=0;k27=0;k28=0;k
29=0;k30=0;
p1=0;p2=0;p3=0;p4=0;p5=0;p6=0;p7=0;p8=0;p9=0;p10=0;p11=0;p12=0;p13=0;p14=0;p15=
0;
p16=0;p17=0;p18=0;p19=0;p20=0;p21=0;p22=0;p23=0;p24=0;p25=0;p26=0;p27=0;p28=0;p
29=0;p30=0;

mse_matrix_tr=[];
mse_matrix_test=[];
mse_matrix=[];
min_mse_matrix_tr=[];
min_mse_matrix_test=[];
CE_matrix_tr=[];
```

```
CE_matrix=[];
CE_train = [];
min_CE_train = [];
CE_test = [];
min_CE_test = [];
CE_train_t =[];
CE_test_t =[];

for epoch=1:80
    epoch
    for i=1:7352
        x1=X_train(i,:);
        z1=x1*transpose(w1);
        h1=tanh(z1);
        h1=[h1 1];
        y1=h1*transpose(w2);
        r1=ReLU(y1);
        r1=[r1 1];
        ow1=r1*transpose(wo);
        o1=softmax(ow1);

        df=encoding(Y_train);
        d=df(i,:);

        e=d-o1;
        lr=0.001;

        sigma_kl=diag(dersoftmax(ow1))*transpose(e);
        delta_wo=lr*(sigma_kl*r1);
        wo=wo+delta_wo;

        sigma_kl2=diag(derReLU(y1))*(wo(:,1:100))'*sigma_kl;
        delta_w2=lr*sigma_kl2*h1;
        w2=w2+delta_w2;

        sigma_kl1=diag(dertanh(z1))*(w2(:,1:200))'*sigma_kl2;
        delta_w1=lr*sigma_kl1*x1;
        w1=w1+delta_w1;

        o_train_max(i)=max(o1,[],2);
        o_train_index(i)=find(o1==o_train_max(i));
    end

    % Calculated cross-entropy error for train data
    CE_train=sum((d.*log(o1)));
    CE_train_t = -(1/7352)*CE_train;
    CE_matrix_tr=[CE_matrix_tr, CE_train_t];
    min_CE_train(epoch)=min(CE_matrix_tr);



    for i=1:2947
        x1=X_test(i,:);
        z1=x1*transpose(w1);
        h1=tanh(z1);
        h1=[h1 1];
        y1=h1*transpose(w2);
        r1=ReLU(y1);
        r1=[r1 1];
```

```
        ow1=r1*transpose(wo);
        o1=softmax(ow1);

        df=encoding(Y_test);
        d=df(i,:);

        et=d-o1;

        o_test_max(i)=max(o1,[],2);
        o_test_index(i)=find(o1==o_test_max(i));
    end
    % Calculated cross-entropy error for test data

    CE_test=sum(d.*log(o1));
    CE_test_t = -(1/2947)*CE_test;
    CE_matrix=[CE_matrix, CE_test_t];
    min_CE_test(epoch)=min(CE_matrix);


end



% Plot the cross-entropy error graph

figure;
plot(1:epoch,min_CE_train, 1:epoch, min_CE_test);
grid;
xlabel('epoch');
ylabel('CE');
title('Figure 1 - Nöron kardeşliği')
legend('CE_train','CE_test');


for j=1:7352
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==1)
                    yurume_tr=yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==2)
                    yukari_yurume_tr=yukari_yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==3)
                    asagi_yurume_tr=asagi_yurume_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==4)
                    oturma_tr=oturma_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==5)
                    ayakta_tr=ayakta_tr+1;
                end
                if(o_train_index(j)==Y_train(j) && o_train_index(j)==6)
                    yatma_tr=yatma_tr+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==1)
                    p1=p1+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==1)
```

```
                    p2=p2+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==1)
                    p3=p3+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==1)
                    p4=p4+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==1)
                    p5=p5+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==2)
                    p6=p6+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==2)
                    p7=p7+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==2)
                    p8=p8+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==2)
                    p9=p9+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==2)
                    p10=p10+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==3)
                    p11=p11+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==3)
                    p12=p12+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==3)
                    p13=p13+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==3)
                    p14=p14+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==3)
                    p15=p15+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==4)
                    p16=p16+1;
```

```
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==4)
                    p17=p17+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==4)
                    p18=p18+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==4)
                    p19=p19+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==4)
                    p20=p20+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==5)
                    p21=p21+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==5)
                    p22=p22+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==5)
                    p23=p23+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==5)
                    p24=p24+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==6 &&
o_train_index(j)==5)
                    p25=p25+1;
                end

                if(o_train_index(j)~=Y_train(j) && Y_train(j)==1 &&
o_train_index(j)==6)
                    p26=p26+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==2 &&
o_train_index(j)==6)
                    p27=p27+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==3 &&
o_train_index(j)==6)
                    p28=p28+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==4 &&
o_train_index(j)==6)
                    p29=p29+1;
                end
                if(o_train_index(j)~=Y_train(j) && Y_train(j)==5 &&
o_train_index(j)==6)
                    p30=p30+1;
                end
            end
```

```
confusionMatrixTraining=[yurume_tr p6 p11 p16 p21 p26;
        p1 yukari_yurume_tr p12 p17 p22 p27;
        p2 p7 asagi_yurume_tr p18 p23 p28;
        p3 p8 p13 oturma_tr p24 p29;
        p4 p9 p14 p19 ayakta_tr p30;
        p5 p10 p15 p20 p25 yatma_tr];



        for j=1:2947
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==1)
                    yurume=yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==2)
                    yukari_yurume=yukari_yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==3)
                    asagi_yurume=asagi_yurume+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==4)
                    oturma=oturma+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==5)
                    ayakta=ayakta+1;
                end
                if(o_test_index(j)==Y_test(j) && o_test_index(j)==6)
                    yatma=yatma+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==1)
                    k1=k1+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==1)
                    k2=k2+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==1)
                    k3=k3+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==1)
                    k4=k4+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==1)
                    k5=k5+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==2)
                    k6=k6+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==2)
                    k7=k7+1;
                end
```

```
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==2)
                    k8=k8+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==2)
                    k9=k9+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==2)
                    k10=k10+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==3)
                    k11=k11+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==3)
                    k12=k12+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==3)
                    k13=k13+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==3)
                    k14=k14+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==3)
                    k15=k15+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==4)
                    k16=k16+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==4)
                    k17=k17+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==4)
                    k18=k18+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==4)
                    k19=k19+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==4)
                    k20=k20+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==5)
                    k21=k21+1;
                end
```

```
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==5)
                    k22=k22+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==5)
                    k23=k23+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==5)
                    k24=k24+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==6 &&
o_test_index(j)==5)
                    k25=k25+1;
                end

                if(o_test_index(j)~=Y_test(j) && Y_test(j)==1 &&
o_test_index(j)==6)
                    k26=k26+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==2 &&
o_test_index(j)==6)
                    k27=k27+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==3 &&
o_test_index(j)==6)
                    k28=k28+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==4 &&
o_test_index(j)==6)
                    k29=k29+1;
                end
                if(o_test_index(j)~=Y_test(j) && Y_test(j)==5 &&
o_test_index(j)==6)
                    k30=k30+1;
                end
            end

confusionMatrixTest=[yurume k6 k11 k16 k21 k26;
        k1 yukari_yurume k12 k17 k22 k27;
        k2 k7 asagi_yurume k18 k23 k28;
        k3 k8 k13 oturma k24 k29;
        k4 k9 k14 k19 ayakta k30;
        k5 k10 k15 k20 k25 yatma];

trainingSuccessRate =
sum(diag(confusionMatrixTraining))/sum(sum(confusionMatrixTraining));
testSuccessRate = sum(diag(confusionMatrixTest))/sum(sum(confusionMatrixTest));

function o = encoding(v)
    % Giriş vektörünün uzunluğunu al
    n = length(v);

    % Çıkış matrisini başlangıçta sıfırlarla doldur
    o = zeros(n, 6);

    % Her eleman için encoding işlemi yap
    for i = 1:n
        switch v(i)
```

```
            case 1
                o(i, :) = [1 0 0 0 0 0];
            case 2
                o(i, :) = [0 1 0 0 0 0];
            case 3
                o(i, :) = [0 0 1 0 0 0];
            case 4
                o(i, :) = [0 0 0 1 0 0];
            case 5
                o(i, :) = [0 0 0 0 1 0];
            case 6
                o(i, :) = [0 0 0 0 0 1];
            otherwise
                error('Geçersiz giriş değeri');
        end
    end
end



function[o] = tanh(v)                       % Activation Function
    o=(exp(2*v)-1)./(exp(2*v)+1);
end

function[a] = ReLU(b)                       % Activation Function
    for i=1:100
        if(b(i)<0)
            a(i)=0;
        elseif(b(i)>=0)
            a(i)=b(i);
        end
    end
end



function[a] = derReLU(b)                       % Activation Function
    for i=1:100
        if(b(i)<0)
            a(i)=0;
        elseif(b(i)>=0)
            a(i)=1;
        end
    end
end

function[o] = dertanh(v)                       % Activation Function
    o=1-(((exp(2*v)-1)./(exp(2*v)+1)).^2);
end

function[o] = softmax(v)                    % Softmax Activation Function
    o = exp(v) / sum(exp(v));
end



function[o] = dersoftmax(v)
%Derivative of softmax activation function
```

```
exp_values = transpose(exp(v));
oo = transpose(repmat(v, 6, 1));

% Matrisin köşegen elemanlarını sıfırla
oo(logical(eye(size(oo)))) = 0;

% o1'ı tanımla
o1 = oo;

os = (o1 * exp_values) / sum(exp_values).^2;
o = transpose(os);
end
```

-Graph of the variation of Cross-Entropy Error (CE) loss with respect to the number of epochs on training and test sets

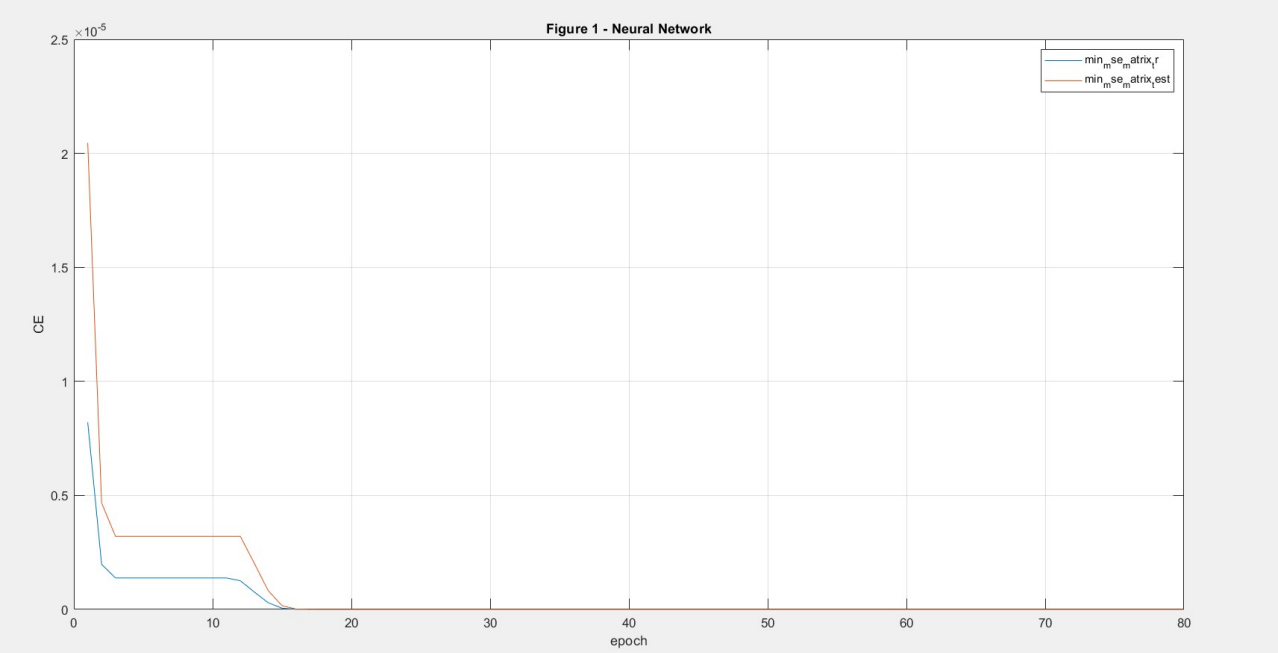


Figure 6:Cross-Entropy error graph

-The minimum loss value obtained and the epoch number at which this value occurred for each set. Starting from the top row and going down, the minimum loss values for all epochs from 1 to 80 are provided below

8.20437181333658e-06	1.97660830426707e-06
1.38079277132623e-06	1.38079277132623e-06
1.38079277132623e-06	1.38079277132623e-06
1.38079277132623e-06	1.38079277132623e-06
1.38079277132623e-06	1.38079277132623e-06
1.38079277132623e-06	1.25594220102133e-06
7.65760936904652e-07	2.93582037436254e-07
5.44162628383850e-08	5.91483239433900e-09
1.14475092284099e-09	3.40249281701208e-10
1.19993387810038e-10	4.70865499951161e-11
2.06603565111355e-11	9.80028332669694e-12
4.45187002295097e-12	2.09415276449394e-12

1.17130291372684e-12	7.98488808517612e-13
6.47660895778783e-13	1.83116005797887e-13
3.47147077216984e-14	6.82215204772192e-15
1.32953771219683e-15	2.55665258489870e-16
5.07323094966878e-17	1.06259098013908e-17
2.56762689734351e-18	7.99840615126051e-19
3.33263829861860e-19	1.88819525204184e-19
1.34366028838832e-19	1.08383737837623e-19
9.17481729505947e-20	7.93216690543465e-20
6.98542730867145e-20	6.25432318035926e-20
5.55512791319442e-20	4.93222664348372e-20
4.35430085207387e-20	3.80021111643847e-20
3.27720816284547e-20	2.82897521431988e-20
2.43138517446231e-20	2.10611710989605e-20
1.80709034662689e-20	1.53843550102671e-20
1.30935529735536e-20	1.11436171705689e-20
9.38873946872850e-21	7.82055765360002e-21
6.52469857048498e-21	5.47072460357996e-21
4.60007997200565e-21	3.85655975620839e-21
3.23437592410803e-21	2.71262862423995e-21
2.28195338136569e-21	1.92080321033217e-21
1.61854849347266e-21	1.36761960481956e-21
1.16096078780062e-21	9.83148317734612e-22
8.30329072539697e-22	7.03544970956760e-22
5.95851737726667e-22	5.06087979392518e-22
4.33666881135688e-22	3.72914997455185e-22
3.32892822747473e-22	2.97708098841017e-22
2.67529168318914e-22	2.41559999593164e-22

-Using the weights obtained at the end of training, the confusion matrix for the training set is as follows:

Variables - confusionMatrixTraining

confusionMatrixTraining

6x6 double

	1	2	3	4	5	6	7
1	1224	2	0	0	0	0	
2	1	1071	1	0	0	0	
3	0	6	980	0	0	0	
4	0	0	0	1269	16	1	
5	0	0	0	21	1353	0	
6	0	0	0	0	0	1407	
7							

-Using the weights obtained at the end of training, the confusion matrix for the test set is as follows:

Variables - confusionMatrixTest

confusionMatrixTest

6x6 double

	1	2	3	4	5	6	7
1	490	4	2	0	0	0	
2	14	449	8	0	0	0	
3	9	27	384	0	0	0	
4	0	1	0	437	53	0	
5	1	0	0	32	499	0	
6	0	0	0	29	0	508	
7							
8							

-The calculated success rate for the training set is as follows:

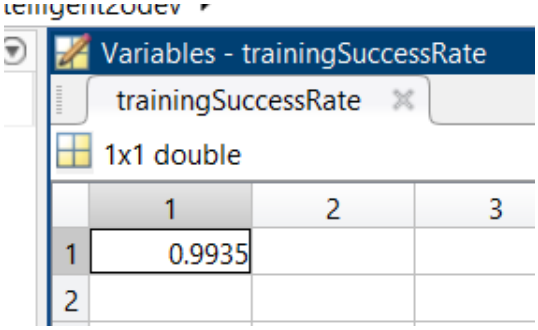


Figure 9:Success Rate of Training

-The calculated success rate for the test set is as follows:

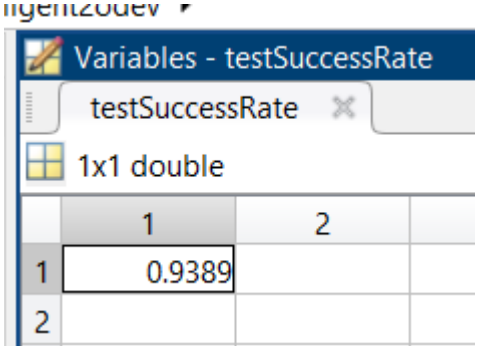


Figure 10:Success Rate of Test

Question 3)

The sigmoid activation function and mean squared error (MSE) cost function are typically preferred for binary classification problems; the sigmoid function facilitates the interpretation of outputs as probabilities by compressing them between 0 and 1. On the other hand, the softmax activation function and cross-entropy (CE) cost function are commonly used for multi-class classification problems; softmax normalizes outputs as probabilities, ensuring that the sum of outputs is 1. While MSE is hindered by the vanishing gradient problem among its drawbacks, CE stands out with its effective handling of classification errors and rapid learning capabilities, although it may be more sensitive to dataset irregularities. The choice depends on the problem context and dataset characteristics.